Helm-chart

Цели работы:

- изучить концепцию Helm Chart и его компоненты
- освоить создание и структуру Helm Chart
- научиться использовать шаблонизацию и переменные
- освоить установку, обновление и удаление приложений с помощью Helm
- понять разницу между прямым применением манифестов и использованием Helm

Helm Chart в Kubernetes — это пакетный формат, используемый для управления приложениями и ресурсами в кластере Kubernetes. Helm является менеджером пакетов для Kubernetes, который позволяет легко развертывать, управлять и обновлять приложения в кластере.

Основные компоненты Helm:

- **Helm CLI** - инструмент командной строки для управления Charts
- **Chart** - пакет, содержащий все ресурсы для развертывания приложения
- **Release** - работающий экземпляр Chart в кластере
- **Repository** - хранилище Charts для общего доступа

Установка Helm Charts:

Windows

Шаг 1. Скачайте Helm

1. Откройте браузер и перейдите по адресу:
   `https://github.com/helm/helm/releases`
2. Найдите последнюю версию

Шаг 2. Установите Helm

1. **Создайте папку** для Helm:
2. **Извлеките** скачанный ZIP-архив в папку, должен появиться файл helm.exe
3. **Добавьте путь в системные переменные**:

– Нажмите Win + R → введите sysdm.cpl → Enter
– "Дополнительно" → "Переменные среды"
– В разделе "Системные переменные" найдите Path → "Изменить"
– "Создать" → введите путь к созданной папке
– Нажмите "OK" во всех окнах

Шаг 3. Проверьте установку

helm version

Linux

Шаг 1. Скачать и выполнить скрипт установки

curl -fsSL -o get_helm.sh
https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3

chmod 700 get_helm.sh

./get_helm.sh

Шаг 2. Проверьте установку

helm version

Пример 1. Создание базового Helm Chart
Шаг 1. Создаем структуру Chart

```
C:\Temp>helm create demo-chart
Creating demo-chart

C:\Temp>cd demo-chart

C:\Temp\demo-chart>tree
Структура папок
Серийный номер тома: E05A-8CFA
C:.
├───charts
└───templates
    └───tests
```

Замечание: на скриншоте приведены команды для командной строки Windows, в Linux другие команды

Что происходит:

– helm create - автоматически генерирует структуру Chart со всеми необходимыми файлами
– Создается директория demo -chart с готовыми шаблонами
– tree . показывает иерархию файлов

Ожидаемый результат:

```
demo-chart/
│
├── Chart.yaml                          # Метаданные о Chart
├── values.yaml                         # Значения по умолчанию для конфигурации
├── charts/                             # Директория для зависимостей Charts
└── templates/                          # Шаблоны Kubernetes ресурсов
    ├── deployment.yaml                 # Шаблон Deployment
    ├── service.yaml                    # Шаблон Service
    ├── hpa.yaml                        # Шаблон Horizontal Pod Autoscaler
    ├── ingress.yaml                    # Шаблон Ingress
    ├── _helpers.tpl                    # Файл определения вспомогательных шаблонов
    └── tests/                          # Директория для тестов Charts
        └── test-connection.yaml        # Тест для проверки доступности
└── .helmignore                   # Файлы для игнорирования (аналог .gitignore)
```

Шаг 2. Изучаем структуру Chart.yaml

Файл: Chart.yaml

```yaml
apiVersion: v2                          # Версия Chart API (v2 для Helm 3)
name: demo-chart                        # Имя Chart (обязательное поле)
description: A Helm chart for Kubernetes  # Описание Chart
type: application                       # Тип: application или library
version: 0.1.0                    # Версия Chart (семантическое версионирование)
appVersion: "1.0.0"                     # Версия приложения (строка)
```

Объяснение полей:

- apiVersion: v2 - указывает, что Chart использует API версии 2 (только для Helm 3)
- name: demo -chart - уникальное имя Chart'a, используется при создании Release
- description - человеко-читаемое описание, что делает этот Chart
- type: application - тип Chart'a:
  - application - для развертывания приложений (по умолчанию)
  - library - для вспомогательных Charts с общими шаблонами
- version: 0.1.0 - версия самого Chart'a, должна следовать семантическому версионированию
- appVersion: "1.0.0" - версия приложения, которое развертывает Chart

Дополнительные поля (можно добавить):

```yaml
keywords:
  - web
  - nginx
  - application
home: https://github.com/myorg/demo-chart         # Домашняя страница
sources:
  - https://github.com/myorg/demo-chart
maintainers:
  - name: Your Name
    email: your@email.com
```

Шаг 3 Настраиваем values.yaml

Файл: ***values.yaml***

```yaml
# Количество реплик Pod'ов
replicaCount: 2

# Настройки образа контейнера
image:
  repository: nginx          # Репозиторий Docker образа
  tag: "1.21"                # Тег образа (версия)
  pullPolicy: IfNotPresent # Политика загрузки: Always, IfNotPresent, Never

# Настройки Service
service:
  type: ClusterIP            # Тип Service: ClusterIP, NodePort, LoadBalancer
  port: 80                   # Порт Service

# Дополнительные настройки (уже есть в сгенерированном Chart)
imagePullSecrets: []         # Секреты для загрузки образов
nameOverride: ""             # Переопределение имени
fullnameOverride: ""         # Переопределение полного имени
```

Как используются значения в шаблонах:

В templates/deployment.yaml: {{ .Values.replicaCount }}

В templates/deployment.yaml: {{ .Values.image.repository }}:{{ .Values.image.tag }}

В templates/service.yaml: {{ .Values.service.type }}

Шаг 4. Просмотр сгенерированных шаблонов

Helm обработает все файлы в папке templates/, подставит значения из values.yaml и выведет готовые YAML-манифесты в терминал или в указанный файл

```
# Генерируем манифесты без установки
helm template demo-chart ./demo-chart

# Или с нужными значениями
helm template demo-chart ./demo-chart --set replicaCount=3
```

Что получаем:

Будет сгенерирован готовый Deployment

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: demo-chart
spec:
  replicas: 2              # Из values.yaml: replicaCount
  template:
    spec:
      containers:
      - image: nginx:1.21  # Из values.yaml: image.repository и image.tag
        name: demo-chart
---
```

И Service

```yaml
apiVersion: v1
kind: Service
metadata:
  name: demo-chart
spec:
  type: ClusterIP          # Из values.yaml: service.type
  ports:
  - port: 80              # Из values.yaml: service.port
    targetPort: 80
```

```yaml
---
# Source: demo-chart/templates/service.yaml
apiVersion: v1
kind: Service
metadata:
  name: demo-chart
  labels:
    helm.sh/chart: demo-chart-0.1.0
    app.kubernetes.io/name: demo-chart
    app.kubernetes.io/instance: demo-chart
    app.kubernetes.io/version: "1.16.0"
    app.kubernetes.io/managed-by: Helm
spec:
  type: ClusterIP
  ports:
    - port: 80
      targetPort: http
      protocol: TCP
      name: http
```

```yaml
# Source: demo-chart/templates/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: demo-chart
  labels:
    helm.sh/chart: demo-chart-0.1.0
    app.kubernetes.io/name: demo-chart
    app.kubernetes.io/instance: demo-chart
    app.kubernetes.io/version: "1.16.0"
    app.kubernetes.io/managed-by: Helm
spec:
  replicas: 1
  selector:
    matchLabels:
      app.kubernetes.io/name: demo-chart
      app.kubernetes.io/instance: demo-chart
  template:
    metadata:
      labels:
        helm.sh/chart: demo-chart-0.1.0
        app.kubernetes.io/name: demo-chart
        app.kubernetes.io/instance: demo-chart
        app.kubernetes.io/version: "1.16.0"
        app.kubernetes.io/managed-by: Helm
```

Таким образом, видно, что Helm использует values.yaml как централизованный файл конфигурации. Изменяя значения в нем, мы меняем поведение всего приложения без редактирования шаблонов.

Пример 2. Установка и управление Release

Шаг 1. Устанавливаем Chart

```
C:\Temp>helm install my-release ./demo-chart
NAME: my-release
LAST DEPLOYED: Thu Nov 20 18:09:19 2025
NAMESPACE: default
STATUS: deployed
REVISION: 1
DESCRIPTION: Install complete
NOTES:
1. Get the application URL by running these commands:
  export POD_NAME=$(kubectl get pods --namespace default -l "app.kubernetes.io/name=demo-c
e=my-release" -o jsonpath="{.items[0].metadata.name}")
  export CONTAINER_PORT=$(kubectl get pod --namespace default $POD_NAME -o jsonpath="{.spe
inerPort}")
  echo "Visit http://127.0.0.1:8080 to use your application"
  kubectl --namespace default port-forward $POD_NAME 8080:$CONTAINER_PORT
```

```
C:\Temp>helm list
NAME            NAMESPACE       REVISION
my-release      default         1
```

Шаг 2. Обновляем конфигурацию

```
C:\Temp>helm upgrade my-release ./demo-chart --set replicaCount=3
Release "my-release" has been upgraded. Happy Helming!
NAME: my-release
LAST DEPLOYED: Thu Nov 20 18:09:44 2025
NAMESPACE: default
STATUS: deployed
REVISION: 2
DESCRIPTION: Upgrade complete
```

Шаг 3. Проверяем состояние

```
C:\Temp>helm status my-release
NAME: my-release
LAST DEPLOYED: Thu Nov 20 18:09:44 2025
NAMESPACE: default
STATUS: deployed
REVISION: 2
DESCRIPTION: Upgrade complete
RESOURCES:
==> v1/ServiceAccount
NAME                    SECRETS   AGE
my-release-demo-chart   0         34s

==> v1/Service
NAME                    TYPE        CLUSTER-IP    EXTERNAL-IP   PORT(S)   AGE
my-release-demo-chart   ClusterIP   10.99.70.52   <none>        80/TCP    34s

==> v1/Deployment
NAME                    READY   UP-TO-DATE   AVAILABLE   AGE
my-release-demo-chart   2/3     3            2           34s

==> v1/Pod(related)
NAME                                     READY   STATUS    RESTARTS   AGE
my-release-demo-chart-678b45f7f8-4ghgc   1/1     Running   0          34s
my-release-demo-chart-678b45f7f8-8w4zj   1/1     Running   0          9s
```

```
C:\Temp>helm list
NAME        NAMESPACE   REVISION   UPDATED                                   STATUS    CHART                 APP VERSION
my-release  default     2          2025-11-20 18:09:44.3455133 +0300 MSK     deployed  demo-chart-0.1.0      1.16.0
```

```
C:\Temp>kubectl get deployments -l app.kubernetes.io/instance=my-release
NAME                    READY   UP-TO-DATE   AVAILABLE   AGE
my-release-demo-chart   3/3     3            3           46s
```

Когда выполняете helm install, происходит следующее:

- Helm генерирует манифесты в памяти
- отправляет их в Kubernetes API
- Kubernetes создает реальные ресурсы в вашем кластере
- манифесты сохраняются в Secrets кластера как история Release

Задания для самостоятельного решения

Задание 11.1. Создание кастомного Helm Chart
Создайте Helm Chart с именем webapp-chart для веб-приложения:

1) используйте образ nginx:1.23
2) настройте 3 реплики приложения
3) создайте Service типа ClusterIP на порту 8080
4) добавьте метки: environment: production, team: backend
5) Настройте лимиты ресурсов: CPU 200m, Memory 256Mi

Проверочные команды:

```
helm install webapp-release ./webapp-chart
helm get manifest webapp-release
kubectl get deployments -l environment=production
kubectl get services -l team=backend
kubectl describe deployment webapp-release-webapp-chart
```

В отчете:

1) ***создание нового Chart***
2) ***просмотр структуры (tree или ls -la)***
3) ***настройка Chart.yaml***

```yaml
apiVersion: v2
name: webapp-chart
description: A Helm chart for web application
type: application
version: 0.1.0
appVersion: "1.23"

# Дополнительные метаданные (опционально)
keywords:
  - web
  - nginx
  - production
maintainers:
  - name: student
    email: student@university.edu
```

4) ***настройка values.yaml (полностью переписываем или редактируем)***

Файл: `values.yaml` ***(полностью переписываем или редактируем)***

```yaml
# Количество реплик
replicaCount: 3

# Настройки образа
image:
  repository: nginx
  tag: "1.23"
  pullPolicy: IfNotPresent

# Настройки Service
service:
  type: ClusterIP
  port: 8080
  targetPort: 80
```

```
# Дополнительные метки
labels:
  environment: production
  team: backend

# Дополнительные аннотации (опционально)
annotations: {}

# Настройки ресурсов
resources:
  limits:
    cpu: 200m
    memory: 256Mi
  requests:
    cpu: 100m
    memory: 128Mi
```

5) *настройка Deployment template*

Файл: `templates/deployment.yaml` ***(находим и редактируем секции)***

```
spec:
  replicas: {{ .Values.replicaCount }}

template:
  metadata:
    labels:
      {{- include "webapp-chart.selectorLabels" . | nindent 6 }}
      # ДОБАВЛЯЕМ СВОИ МЕТКИ:
      environment: {{ .Values.labels.environment }}
      team: {{ .Values.labels.team }}

containers:
  - name: {{ .Chart.Name }}
    image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
    imagePullPolicy: {{ .Values.image.pullPolicy }}
    ports:
      - name: http
        containerPort: 80
        protocol: TCP
```

6) *настройка Service template*

Файл: `templates/service.yaml` ***(проверяем настройки портов)***

```
apiVersion: v1
kind: Service
metadata:
  name: {{ include "webapp-chart.fullname" . }}
  labels:
    {{- include "webapp-chart.labels" . | nindent 4 }}
    # ДОБАВЛЯЕМ МЕТКИ В SERVICE ТОЖЕ:
    environment: {{ .Values.labels.environment }}
    team: {{ .Values.labels.team }}
spec:
  type: {{ .Values.service.type }}
  ports:
    - port: {{ .Values.service.port }}        # Это порт Service (8080)
      targetPort: {{ .Values.service.targetPort }} # Это порт контейнера (80)
```

```
      protocol: TCP
      name: http
  selector:
    {{- include "webapp-chart.selectorLabels" . | nindent 4 }}
```

7) *проверка Chart*

```
# Проверяем синтаксис
helm lint ./webapp-chart

# Просматриваем сгенерированные манифесты
helm template webapp-demo ./webapp-chart

# Или с подробным выводом
helm template webapp-demo ./webapp-chart -debug
```

8) *установка и проверка*

```
# Устанавливаем Chart
helm install webapp-release ./webapp-chart

# Проверяем установку
helm list

# Проверяем созданные ресурсы
kubectl get deployments -l environment=production
kubectl get services -l team=backend
kubectl get pods -l app.kubernetes.io/name=webapp-chart
```

9) *проверочные команды:*

```
# 1. Проверяем установленный Release
helm get manifest webapp-release

# 2. Проверяем Deployment с метками
kubectl get deployments -l environment=production

# 3. Проверяем детали Deployment
kubectl describe deployment webapp-release-webapp-chart

# 4. Проверяем Service
kubectl get service webapp-release-webapp-chart
kubectl describe service webapp-release-webapp-chart

# 5. Проверяем Pods
kubectl get pods -l environment=production
```

10) ожидаемый результат:

Deployment должен иметь:

- 3 реплики
- Образ: nginx:1.23
- Метки: environment: production, team: backend

Service должен иметь:

- Тип: ClusterIP
- Порт: 8080
- Метки: environment: production, team: backend

Задание 11.2. Шаблонизация с условиями и циклами
Модифицируйте Chart из задания 11.1, добавив расширенную шаблонизацию:
1) Добавьте параметр extraLabels в values.yaml для дополнительных меток
2) Добавьте параметр environmentVariables для кастомных переменных окружения
3) Используйте циклы range для динамического создания меток и переменных окружения
4) Добавьте условие для включения/отключения меток

values.yaml должен включать:

```
extraLabels:
  project: "university"
  course: "kubernetes"
  semester: "spring"

environmentVariables:
  - name: "APP_ENV"
    value: "production"
  - name: "APP_DEBUG"
    value: "false"
  - name: "LOG_LEVEL"
    value: "info"
```

Проверочные команды:
```
helm template webapp ./webapp-chart --debug
helm upgrade webapp-release ./webapp-chart --set extraLabels.semester="autumn"
kubectl get pods -l project=university
kubectl exec <pod-name> -- env | grep APP_
```

Задание 11.3. Многофайловая конфигурация и ConfigMap
Расширьте Chart, добавив управление конфигурацией через разные environment файлы:
1) Создайте ConfigMap с настройками приложения
2) Создайте два файла values: values-development.yaml и values-production.yaml
3) Настройте разные параметры для development и production окружений
4) Добавьте ConfigMap в Deployment как volume или переменные окружения

Структура

```
webapp-chart/
├── values-development.yaml
├── values-production.yaml
└── templates/
    └── configmap.yaml
```

Параметры для разных окружений:

- Development: 1 реплика, debug логи, development метка
- Production: 3 реплики, info логи, production метка

Проверочные команды:
```
# Тестируем development конфигурацию
helm install webapp-dev ./webapp-chart -f values-development.yaml
kubectl get configmaps
kubectl describe configmap webapp-dev-webapp-chart-config
```

```
# Тестируем production конфигурацию
helm install webapp-prod ./webapp-chart -f values-production.yaml
kubectl get deployments -l environment=production
```

Задание 11.4. Диагностика и отладка Helm Chart

Создайте проблемный Chart и найдите/исправьте ошибки:

Шаг 1: Создайте Chart с преднамеренными ошибками:

- Ошибка в имени параметра (опечатка в values.yaml)
- Синтаксическая ошибка в шаблоне
- Отсутствующая переменная

Шаг 2: Используйте инструменты диагностики:

- helm lint - проверка синтаксиса Chart
- helm template --debug - отладка шаблонов
- helm install --dry-run - тестовая установка

Шаг 3: Исправьте ошибки и проверьте работоспособность

Пример проблемного кода:

```
# values.yaml (с ошибкой)
replicasCount: 3  # ОШИБКА: должно быть replicaCount

# templates/deployment.yaml (с ошибкой)
spec:
  replicas: {{ .Values.replicasCount }}  # ОШИБКА: значения не существует
  template:
    metadata:
      labels:
        app: {{ .Chart.Name }}
         # ОШИБКА: пропущена закрывающая скобка
```

Проверочные команды:

```
# Проверяем синтаксис
helm lint ./problem-chart

# Отладка шаблона
helm template test-release ./problem-chart --debug

# Тестируем установку
helm install test-release ./problem-chart --dry-run

# После исправлений проверяем работоспособность
helm install fixed-release ./fixed-chart
kubectl get pods -l app.kubernetes.io/instance=fixed-release
```

Контрольные вопросы

1. В чем основное преимущество Helm перед прямым применением kubectl apply?
2. Какие основные файлы входят в структуру Helm Chart?
3. Как работает шаблонизация в Helm?
4. В чем разница между Chart и Release?
5. Как можно передать пользовательские значения при установке Chart?
6. Что такое зависимости в Helm?
7. Как можно отладить Helm Chart перед установкой?
8. Какие команды используются для управления жизненным циклом Helm Release?